

ECE 150 Project – Square Root Calculator

Marmoset Tests

Zhifan Li

September 2026

1 Learning objectives

Write a C++ program that uses **Newton’s method** to approximate the square root of a number n , given a starting guess s and a maximum error tolerance e . Read $n \geq 0$, $s > 0$, and $e > 0$ in that order; repeat the relevant prompt until its input is valid.

For $n > 0$, set $x_0 = s$ and repeatedly calculate

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{n}{x_k} \right).$$

Stop at the first update satisfying $|x_{k+1} - x_k| \leq e$ and output x_{k+1} . For $n = 0$, read and validate all three inputs, then output zero directly. This project practises input validation, floating-point arithmetic, loops, and convergence criteria.

2 Public tests /7

Each public test is worth one mark. In the proposed configuration, all tests are blocking tests: release tests run only after all pass.

Test name	Test description
P100_explicit_return	Check for an explicit <code>return 0</code> ; on normal completion of <code>main</code> . This is a course convention, not a C++ requirement.
P101_valid_libraries	Allow only <code><iostream></code> , <code><iomanip></code> , <code><cmath></code> , and <code><cassert></code> . Check that no library root or power routine replaces the required iteration.
P102_check_namespace	Require qualified standard-library names, such as <code>std::cin</code> ; do not use <code>using</code> declarations or directives.
P103_check_endl	Require <code>std::endl</code> for output line endings as the chosen project convention.
P104_using_arrays	Use scalar variables; do not use arrays or containers for the calculation.
P105_basic_input	Read $(n, s, e) = (9, 3, 0.001)$ in order and terminate after one result. No invalid input is included.
P106_basic_output	For $(n, s, e) = (9, 3, 0.001)$, report 3 using the required result label.

3 Release tests /13

Each release test is worth one mark and may contain multiple runs; all runs must pass. The reference implementation starts at the supplied s and returns the *first new estimate* satisfying the stated stopping rule. Tests compare with that estimate r , not only with $\sqrt{n}$, so the starting guess and tolerance affect the expected answer. Require a finite, non-negative result z with

$$|z - r| \leq 10^{-12} \max(1, |r|).$$

This comparison allowance is separate from the user-supplied e . For $n = 0$, require numerical zero. Print 17 significant digits to preserve `double` precision.

Test name	Test description
R100_zero	Use (0, 1, 0.001) and (0, 5, 0.01). Read all inputs and return zero without division by zero.
R101_exact_guess	Use (1, 1, 0.001) and (25, 5, 10^{-6}). The first update is unchanged and must terminate.
R102_guess_above	Use (9, 10, 10^{-6}) and (16, 20, 10^{-6}). Verify convergence from a guess above the root.
R103_guess_below	Use (9, 1, 10^{-6}) and (16, 0.5, 10^{-6}). Verify convergence after the first update overshoots the root.
R104_decimal_input	Use (2.25, 2, 10^{-6}) and (6.25, 1.5, 10^{-6}). Preserve fractional values in both n and s .
R105_below_one	Use (0.25, 1, 10^{-6}) and (0.04, 0.1, 10^{-6}). Test roots and starting guesses below one.
R106_non_squares	Use (2, 1, 10^{-8}), (3, 2, 10^{-8}), and (10, 3, 10^{-8}). Detect integer arithmetic or rounding.
R107_tolerance	For $n = 2$, $s = 1$, test $e = 0.5$, 0.1 , and 10^{-8} . The first two results must be 1.5 and approximately 1.4166666666666667.
R108_guess_used	For $n = 9$, $e = 10$, test $s = 1$, $s = 3$, and $s = 6$. The first-update results must be 5, 3, and 3.75, respectively.
R109_invalid_n	Supply -9 , -0.25 , then 9 for n ; follow with $s = 3$, $e = 0.001$. Repeat only the radicand prompt.
R110_invalid_s	Accept $n = 9$; supply 0, -2 , then 3 for s ; finish with $e = 0.001$. Preserve the accepted n .
R111_invalid_e	Accept $n = 2$, $s = 1$; supply 0, -0.1 , then 0.1 for e . Preserve n and s and use the valid tolerance.
R112_mixed_trials	Combine repeated invalid inputs across all three prompts with valid cases such as (10^{-8} , 1, 10^{-10}) and (10^6 , 10, 10^{-6}).

The public and release tests contribute $7 + 13 = 20$ marks.

4 Notes and known issues

Meaning of the tolerance. In this project, e is an absolute tolerance on the change between successive estimates: $|x_{k+1} - x_k| \leq e$. Do not replace this rule with the squared residual $|x_{k+1}^2 - n| \leq e$, a relative tolerance, or a fixed iteration count. Always output the new estimate, including when the first update meets the condition.

Algorithm requirements. Use `double` for n , s , e , and the estimates. Use the supplied starting guess; do not replace it with 1 or n . Implement the Newton update using arithmetic and a loop. Do not use `sqrt`, `pow`, or equivalent root-solving routines. `std::abs` or `std::fabs` may be used to measure the change.

Required input and output. Use the prompts below, one line per prompt, ending with `std::endl`. Read one value after each prompt. The numeric input lines in this transcript are user input, not program output.

```
Enter a non-negative number:
2
Enter a positive starting guess:
1
Enter a positive error tolerance:
0.1
Square root: 1.4166666666666667
```

Here, the first update gives 1.5 and changes the estimate by $0.5 > 0.1$. The second update changes the estimate by approximately $0.0833333 \leq 0.1$, so the program stops and prints the new estimate shown above.

Invalid input. Reject $n < 0$, $s \leq 0$, and $e \leq 0$. Repeat only the corresponding prompt, with no additional error message. Keep previously accepted values. For example, after accepting $n = 9$, guesses 0, -2 , 3 cause three starting-guess prompts, followed by the tolerance prompt. Validate s and e even when $n = 0$.

Output precision. Print one result line using the label `Square root:` and `std::setprecision(17)`. Equivalent decimal or scientific notation is acceptable. Do not add iteration logs, extra numbers, or blank lines. The label denotes an approximation and does not imply an exact root.

Test-harness design. Parse the number after `Square root:`. Check prompt order and repetition as well as the numerical result. Include equality in the stopping condition: $(n, s, e) = (2, 1, 0.5)$ must stop after one update. Source checks should ignore comments and string literals; output checks alone cannot establish use of Newton's method. A generous timeout detects non-termination; speed is not scored.

Floating-point scope. Tests use finite numeric tokens and values for which the Newton update remains finite and the specified stopping rule is attainable in `double`. Malformed input, early end-of-file, NaN, infinity, overflow, and tolerance values that prevent convergence because of rounding are outside this draft's scope. The test-comparison allowance accounts for ordinary numerical and output rounding; it does not replace e .

5 Secret tests /8

INSTRUCTOR COPY – NOT VISIBLE TO STUDENTS
Exclude this entire page from all student-facing copies.

Each secret test is worth one mark and may contain multiple runs; all runs must pass.

Test name	Test description and expected behaviour
S100_exact_boundary	Use $(n, s, e) = (16, 2, 3)$. The first update is 5 and the change is exactly 3; return 5. Detects using $<$ instead of $\leq$.
S101_new_estimate	Use $(16, 2, 4)$. Return 5, not the starting guess 2 or the exact root 4. Detects returning the old estimate or ignoring the required stopping point.
S102_residual_trap	Use $(100, 5, 3)$. Updates are 12.5, 10.25: the second change is $2.25 \leq 3$, so return 10.25. Its squared residual is $5.0625 > 3$; a residual-based rule would incorrectly continue.
S103_guess_sensitivity	Use $n = 25$, $e = 20$, and guesses 1, 5, and 10. Expect first-update results 13, 5, and 6.25. Detects an internally substituted guess.
S104_tolerance_used	Use $n = 16$, $s = 2$, first with $e = 3$ and then with $e = 1$. Expect 5 and approximately 4.1. Detects a hard-coded tolerance or iteration count.
S105_small_scale	Use $(10^{-8}, 1, 10^{-10})$. Compare against the iterative oracle, with the published allowance. Detects zero shortcuts and coarse stopping rules.
S106_mixed_recovery	Supply tokens $-1, 0, 0, -3, 2, 0, -0.01, 0.1$. Accept $n = 0$, $s = 2$, $e = 0.1$ after 2, 3, and 3 prompts, respectively; return zero only after validating all inputs.
S107_private_batch	Using a private fixed seed, generate 40 triples with $n = 10^u$, $s = 10^v$, $e = 10^w$; sample $u \in [-8, 6]$, $v \in [-3, 4]$, $w \in [-10, -1]$ uniformly. Compare each run against the iterative oracle; prepend invalid values in every fifth run.

Reference and reproducibility. Record the generated triples, seed, and oracle outputs in the instructor's test fixture. For positive n , apply the same `double` update expression as the assignment and stop at the first qualifying change. For zero, return zero after input validation. Verify termination and finite intermediates before adding a generated case. Avoid generated cases whose stopping decision depends on rounding at the tolerance boundary; S100 already tests equality using exactly representable values.

Visibility. Use the separate student PDF for distribution. This label is a reminder, not access control: the instructor PDF and this LaTeX source contain the secret tests. Compile with `\studentversion` defined to omit this page.